

Reproducibility

Reproducibility I

Reproducibility in computational research has well-documented prerequisites: open data, open code, and a deterministic build that can be re-run from scratch [peng2011reproducible]. The bundled `manuscript/config.yaml` is intentionally configured to satisfy all three for **strict reproducibility**:

1. `search.sources`: `[local]` consumes `data/corpus.json`, which is a curated and committed JSON corpus. No network is required to run the pipeline.
2. `search.cache_dir`: `output/search/cache` writes deterministic JSON cache files; running the same query twice produces a byte-identical artifact tree (modulo timestamp metadata in the cache file itself).
3. `enrichment.fetch_abstracts`: `true` reads abstracts directly from the corpus when present; no network fetch is required.

Reproducibility II

4. `enrichment.fetch_fulltext`: `false` is the default — full-text fetching is opt-in and gated behind the optional `pypdf` dependency (`uv sync --group rendering`).
5. `llm.enabled`: `false` is the default — the LLM stage is opt-in and requires a running `ollama serve`. When enabled, `seed: 42` and `temperature: 0.0` are pinned.

Switching to live search I

Replace the sources list with the desired backend set:

```
search:  
  query: "your topic"  
  sources: [arxiv, crossref]  
  crossref_mailto: "you@example.org"
```

A first live run populates output/search/cache/. Commit that directory to the repo and the pipeline becomes reproducible across machines without further configuration changes.

Determinism guarantees I

The full determinism contract is itemised in [tbl:determinism]: every pipeline stage is annotated as fully, conditionally, or non-deterministic, with an explicit mechanism column so reviewers can audit each row independently.

Table 1: Determinism contract by pipeline stage. Cached stages are byte-stable across reruns; live stages depend on the upstream source and are pinned to the cache file once a successful run completes.

Stage	Deterministic?	Mechanism
Search (cached hit)	yes	SearchCache JSON files
Search (cache miss)	no	live API
Dedup / merge	yes	DOI / arXiv-id canonical keys; tie-break by score then year unicode folding + stop-word skip; collision suffix is deterministic
Citation-key generation	yes	byte-stable format pinning (verified by tests/infra_tests/reference/)
BibTeX writer	yes	per-paper <safe_id>.txt cache
Abstract fetch	yes (cached) / no (live)	per-paper <safe_id>.{pdf,txt}
Fulltext fetch	yes (cached) / mostly (live)	cache; live fetch's pypdf text extraction is not bit-stable across versions
LLM synthesis	mostly	seed=42, temperature=0.0; Ollama deterministic up to its own minor variance

Determinism guarantees II

Figure generation

yes (within Matplotlib version)

fixed palette, fixed bin width, no
random subsampling

Limitations I

The reproducibility contract enumerated in [`@sec:reproducibility`] (items 1–5 and [`@tbl:determinism`]) does not eliminate the following well-defined sources of non-reproducibility, which are surfaced here so reviewers can audit them explicitly rather than inferring from the contract table:

- ▶ **Live search drift.** When `config.search.sources` includes `arxiv` or `crossref`, the first cache-miss invocation hits the live API; the cached JSON freezes that response, but two cold-start clones running on different days will see different paper sets. Pin a `LocalBackend` corpus or commit `output/search/cache/` to break this dependency.

Limitations II

- ▶ **pypdf version drift.** The fulltext fetcher uses pypdf to extract text from a downloaded PDF. pypdf's text-extraction algorithm is not bit-stable across major versions; upgrading pypdf can produce different `<safe_id>.txt` cache contents from the same source PDF. The PDF bytes themselves are bit-stable so the cache freezes the inputs, not the extraction.
- ▶ **Ollama version drift.** Pinning `seed=42` and `temperature=0.0` controls Ollama's sampling, but the model weights, tokenizer, and template can change between Ollama releases. Document the Ollama version alongside `config.llm.model` when archiving a run for replication.
- ▶ **Paperclip backend status.** The paperclip backend is opt-in and currently degrades to HTTP 405 on the production endpoint; the run records the error in `SearchResult.errors[paperclip]` and continues. Treat paperclip results as advisory until the upstream service stabilises.

Limitations III

- ▶ **External backend behaviour outside this project's control.** arXiv and Crossref are the source of truth; this project is faithful to whatever they return. A retraction, metadata fix, or DOI assignment upstream will alter the cache on the next cold-start invocation.

These limitations bound *what* the cache + seed + corpus pinning achieves. Inside those bounds, the contract in [tbl:determinism] is total: every cached pipeline stage is byte-stable across reruns (verified by `tests/test_pipeline.py::TestRunLiteraturePipeline::test_bibtex_byte_identical_across_reruns`).